EXP-Bench: Can AI Conduct AI Research Experiments?

Anonymous Authors

Abstract

Evaluating whether AI agents can conduct rigorous, end-to-end scientific experiments—not merely generate code or summarize papers—is a critical open challenge. We introduce EXP-Bench, a benchmark of 461 tasks drawn from 51 NeurIPS and ICLR 2024 papers (totaling 12,737 fine-grained subtasks) that evaluates AI agents on complete AI research experiments spanning computer vision, NLP, reinforcement learning, and applications. Each task provides a research question and high-level methodology grounded in a peer-reviewed publication, together with a masked repository; ground truth includes a structured design specification, a git-diff implementation, and an execution-validated conclusion. Evaluation uses a conjunctive multi-metric protocol covering Design (D), Implementation (I), Execution (E), and Conclusion (C), with an LLM judge for D/I/C and a Docker-based execution validator for E. Across seven agent–model configurations, agents achieve partial correctness on individual metrics but rarely complete full experiments end-to-end: the best setup (OpenHands + Claude-3.7 Sonnet) reaches only 0.4% on the joint metric requiring correct design, implementation, successful execution, and correct conclusion (All·E✓). Our failure taxonomy identifies missing implementation components, environment errors, and missing conclusions as the dominant bottlenecks. EXP-Bench provides a high-fidelity, execution-grounded testbed revealing that current AI agents handle parts of the research workflow but fall far short of reliably completing end-to-end AI research experiments.

1 Introduction

The vision of AI systems that can autonomously conduct scientific research has catalyzed significant interest across the machine learning community [20, 13, 2, 15]. Recent frontier language models demonstrate impressive capability on isolated scientific tasks—writing literature reviews [16], generating research hypotheses [1], and producing complete papers [20, 29, 22]. Yet the question of whether agents can reliably conduct rigorous, executable experiments—the empirical backbone of AI research—remains largely unanswered.

Existing benchmarks evaluate isolated skills: code generation [25, 12], machine-learning engineering [11, 4, 28, 21], data analysis [7, 10], and research replication [23]. While each captures an important dimension, none evaluates the *complete experimental workflow* that a researcher would follow: formulating a design, writing and integrating code, executing it in a real environment, and interpreting the numerical outcome.

We introduce **EXP-Bench**, a benchmark that evaluates AI agents on *complete, executable AI research experiments* extracted semi-automatically from peer-reviewed publications. Our key insight is that real papers and their open-source code provide both a realistic research context and execution-verifiable ground truth—enabling rigorous, objective evaluation at scale. Concretely, EXP-Bench comprises **461 tasks** from **51 NeurIPS/ICLR 2024 papers** (12,737 fine-grained subtasks) spanning computer vision (CV), natural language processing (NLP), reinforcement learning (RL), and applications. An overview is depicted in Figure 1.

Preprint. Under review.

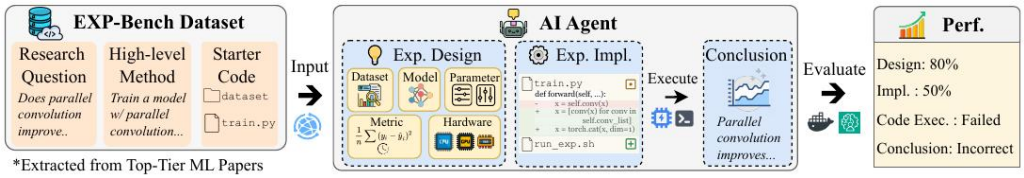

Figure 1: EXP-Bench evaluates AI agents on research experiment tasks extracted semi-autonomously from peer-reviewed AI papers.

Each task provides: (i) a research question derived from the paper, (ii) a high-level experimental methodology, (iii) a masked code repository (with answer-revealing files removed), and (iv) ground truth consisting of a structured design specification, a git-diff implementation, and an execution-validated conclusion. Evaluation uses a *conjunctive* multi-metric protocol: Design (D), Implementation (I), Execution (E), and Conclusion (C)—individually scored by an LLM judge and an execution validator, and jointly combined into strict aggregate metrics (All✓ $= D \wedge I \wedge C$; All·E✓ $= D \wedge I \wedge E \wedge C$).

We benchmark seven agent–model configurations using OpenHands [27] and an IterativeAgent across five frontier LLMs: o3-mini, Claude-3.7 Sonnet, Claude-3.5 Haiku, DeepSeek R1, and Amazon Nova Pro. Our findings reveal a stark gap between partial and complete performance: while individual metrics reach up to 35% (Implementation, Claude-3.7 Sonnet), the joint metric All·E✓ never exceeds 0.5% across all configurations.

Our main contributions are:

- **EXP-Bench**: the first benchmark evaluating AI agents on complete, executable AI research experiments, grounded in peer-reviewed NeurIPS/ICLR 2024 papers.
- A **semi-automated construction pipeline** that extracts structured tasks and implementations from paper PDFs and code repositories, including execution-based verification.
- A **rigorous conjunctive evaluation protocol** combining LLM judging, Docker-based execution validation, and an integrity monitor, along with a detailed failure taxonomy.
- **Empirical findings** across seven agent configurations demonstrating the difficulty of end-to-end experimentation and identifying key failure modes.

2 Related Work

AI for Science and Research Automation. A growing body of work explores AI assistance in the research process [2, 13, 3]. Systems like The AI Scientist [20], Dolphin [29], and Agent Laboratory [22] attempt end-to-end research automation including idea generation, coding, and paper writing. SciAgents [6] uses multi-agent graphs for scientific discovery. However, these systems focus on generating new research rather than evaluating experimental rigor on established tasks. Our work instead uses real papers as ground truth, enabling objective evaluation of whether agents can replicate and execute specific experiments.

Benchmarks for AI Research Agents. MLAgentBench [11] evaluates agents on improving ML models (e.g., on CIFAR-10), while MLE-bench [4] tests ML engineering on Kaggle-style challenges. RE-Bench [28] provides research engineering environments with human expert baselines. MLGym [21] introduces a gym-style framework for AI research tasks. PaperBench [23], the closest prior work, evaluates replication of ICML 2024 papers from scratch, finding that the best agent achieves 21% average replication score. Curie [14] evaluates agents on 46 computer-science research questions. EXP-Bench differs from all of these by providing *execution-grounded* ground truth from papers and code, a *conjunctive* multi-phase evaluation, and a *semi-automated* construction pipeline that scales to 461 tasks.

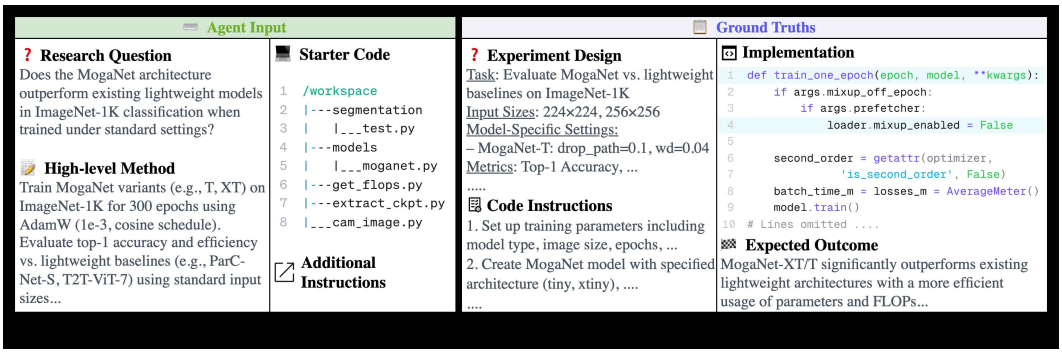

Figure 2: One AI research task example from ICLR 2024 MogaNet [17].

Code Generation and Execution Benchmarks. SWE-bench [12] evaluates agents on GitHub issue resolution. SciCode [25] tests research coding across scientific domains. RepoBench [18] benchmarks repository-level code completion.

Scientific QA and Data Analysis. BLADE [7] evaluates agents on open-ended data science research questions. InfiAgent-DABench [10] and DS-Agent [8] target data analysis tasks. SciBench [26] and SciEval [24] evaluate scientific problem-solving capabilities of LLMs. AgentBench [19] provides a diverse suite of agent evaluation tasks.

3 The EXP-Bench Benchmark and Dataset

3.1 Overview and Task Structure

EXP-Bench evaluates whether AI agents can complete the full lifecycle of an AI research experiment starting from a research question and masked code repository. Each task instance in EXP-Bench is structured as follows:

- **Research Question:** A specific, well-scoped question derived from a peer-reviewed NeurIPS or ICLR 2024 paper (e.g., "How does MogaNet's multi-order gated aggregation affect classification accuracy on ImageNet?").
- **High-Level Methodology:** A description of the experimental approach, including relevant datasets, metrics, and constraints, without revealing implementation details.
- **Masked Repository:** A version of the paper's open-source code repository with answer-revealing files removed (e.g., README sections, scripts that directly produce the result).
- **Ground Truth:** A structured design specification, a git-diff implementation, and an execution-validated conclusion used for evaluation.

An example task from ICLR 2024 MogaNet [17] is shown in Figure 2. The dataset spans four ML research categories—CV, NLP, RL, and Applications—as illustrated in Figure 3.

3.2 Dataset Statistics

EXP-Bench comprises **461 tasks** derived from **51 NeurIPS and ICLR 2024 papers**, yielding a total of **12,737 fine-grained subtasks**. Papers were selected based on citation counts, repository activity (stars/forks), and the availability of runnable code. The dataset spans:

- **Computer Vision (CV):** image classification, detection, segmentation, and generative models.
- **Natural Language Processing (NLP):** language modeling, reasoning, and text generation.
- **Reinforcement Learning (RL):** policy optimization, multi-agent RL, and environment benchmarking.
- **Applications:** domain-specific tasks including anomaly detection, molecular modeling, and time series.

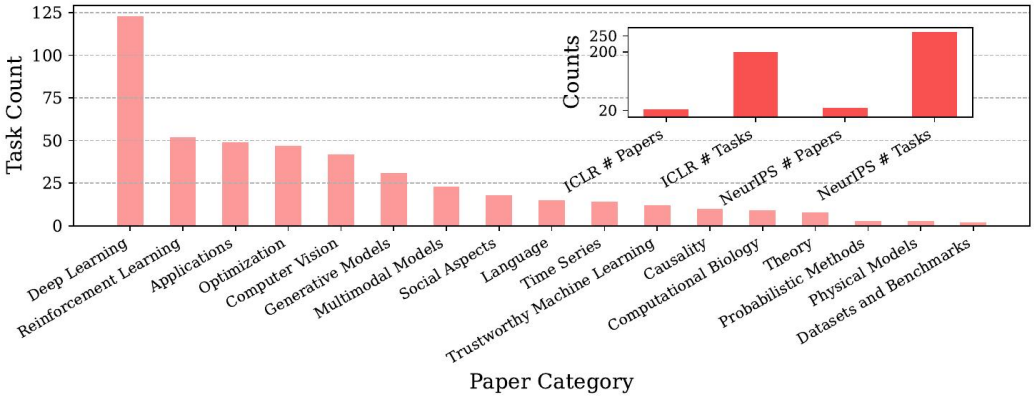

Figure 3: EXP-Bench's dataset comprises tasks from a diverse set of ML research categories.

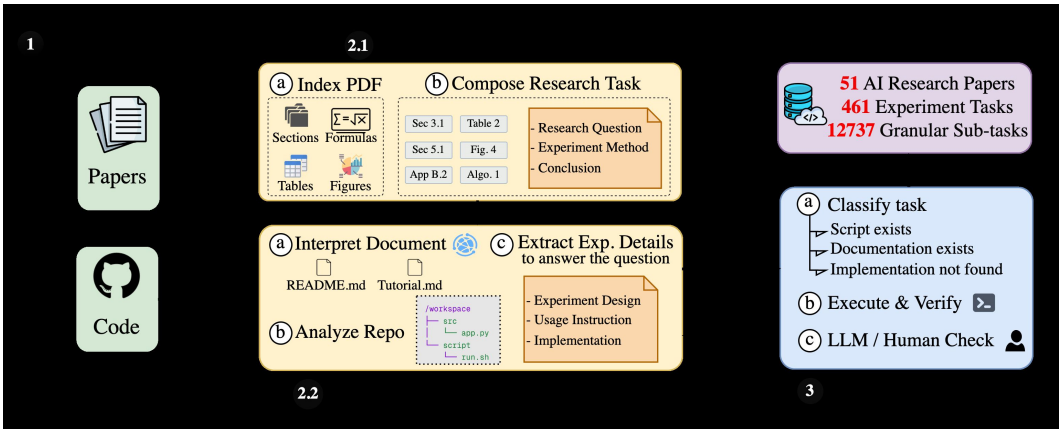

Figure 4: EXP-Bench semi-automated dataset construction pipeline.

3.3 Semi-Automated Construction Pipeline

EXP-Bench employs a three-stage semi-automated pipeline to construct tasks from papers and their code repositories. The pipeline is illustrated in Figure 4.

Stage 1: Source Selection. We identify influential NeurIPS/ICLR 2024 papers with publicly available repositories using citation signals and repository activity metrics (stars, forks). Papers must have runnable code with clearly documented experimental scripts, enabling execution-based verification of tasks.

Stage 2: Experiment Procedure Extraction. *Task Extraction (Stage 2.1):* Tasks are extracted from paper PDFs via OCR and multimodal parsing of figures and tables. We use retrieval-augmented, multi-pass prompting to assemble research questions, experimental methodologies, datasets, metrics, and constraints from each paper. Cross-document re-querying recovers details that may be scattered across the main text, appendix, and supplementary materials.

Implementation Extraction (Stage 2.2): A tool-augmented agent (with access to PDF reader, shell, and web tools) locates relevant scripts, dependencies, and model checkpoints within each repository. The agent outputs runnable script chains with usage instructions. Abstract Syntax Tree (AST) tracing converts these scripts into step-by-step implementation requirements, forming the ground-truth implementation specification as a git-diff.

Stage 3: Verification and Refinement. Candidate tasks are executed in clean Docker containers (Ubuntu 24.04, 4×A40 GPUs, 40-minute timeout) and compared against expected outputs from the

paper. Failures trigger iterative refinement. A lightweight human review cross-checks structured task content for quality. Finally, answer-revealing files (e.g., README sections, result-producing scripts) are masked via scripted git operations to prevent agents from shortcutting the task.

4 Evaluation

4.1 Evaluation Protocol

EXP-Bench employs a multi-metric evaluation framework that assesses four phases of the experimental workflow:

- **Design (D):** Does the agent produce a correct experimental design matching the ground-truth specification? Scored by an o3-mini LLM judge.
- **Implementation (I):** Does the agent's code correctly implement the experimental procedure? Scored by an o3-mini LLM judge comparing agent code to the ground-truth git-diff.
- **Execution (E):** Does the agent's code execute without errors within the 40-minute time limit? Validated by running agent-produced code in Docker containers.
- **Conclusion (C):** Does the agent correctly interpret and report the experimental results? Scored by an o3-mini LLM judge.

Beyond individual metrics, we define *conjunctive* aggregate metrics:

- **I·E:** Implementation correct *and* execution succeeds.
- **All✓:** $D \wedge I \wedge C$ (all LLM-judged phases correct).
- **All·E✓:** $D \wedge I \wedge E \wedge C$ (all phases correct including execution).

An **integrity monitor** runs alongside each agent to detect disallowed behaviors, including accessing the original paper, reverting masked files via `git checkout`, or fabricating output data. Only monitor-passing attempts are included in the final scores.

4.2 Agents and Models

We evaluate two agent scaffolds:

- **OpenHands (OH)** [27]: a general-purpose software agent platform supporting code writing, shell interaction, and file editing.
- **IterativeAgent (IA)**: a simpler iterative agent that alternates between code generation and execution feedback.

Each scaffold is paired with frontier LLMs: **o3-mini**, **Claude-3.7 Sonnet**, **Claude-3.5 Haiku**, **DeepSeek R1**, and **Amazon Nova Pro**, yielding seven agent–model configurations. All experiments are run in isolated Docker environments.

4.3 Main Results

Table 1 reports average benchmark scores across all agent–model pairs. The results reveal several key patterns.

Partial but not complete. All agents achieve non-trivial scores on individual metrics but collapse under conjunctive evaluation. The best-performing configuration, OpenHands + Claude-3.7 Sonnet, achieves I=35.0% and E=33.2%, but All·E✓ drops to only 0.4%. This stark contrast confirms that partial correctness on individual phases does not translate to end-to-end experimental success.

Strong implementation does not imply execution. OpenHands + Claude-3.5 Haiku achieves I=26.2% but only E=9.3%, with I·E=1.3%, indicating that syntactically correct implementations frequently fail to execute due to environment configuration issues. Conversely, OpenHands + Claude-3.7 Sonnet achieves better alignment between I and E (35.0% vs. 33.2%), suggesting this model is better at producing environment-aware code.

Table 1: Average benchmark scores for various models when tested against various evaluation metrics. Popular Agents and LLMs perform poorly on EXP-Bench, showcasing its difficulty.

Agent	Model	D	I	E	I·E	C	All✓	All·E✓	#E
OpenHands	o3-mini	18.4	20.3	15.0	2.9	21.0	1.4	0.5	420
OpenHands	Claude-3.7 Sonnet	16.0	35.0	33.2	14.9	13.4	0.7	0.4	235
OpenHands	Amazon Nova Pro	18.2	19.5	26.8	0.0	15.7	0.0	0.0	56
OpenHands	Claude-3.5 Haiku	20.6	26.2	9.3	1.3	13.8	0.0	0.0	237
OpenHands	DeepSeek R1	6.8	10.0	0.7	0.0	2.4	0.0	0.0	140
IterativeAgent	Claude-3.5 Haiku	6.4	20.6	25.2	5.4	2.2	0.0	0.0	111
IterativeAgent	Amazon Nova Pro	0.1	10.0	18.1	0.0	0.3	0.0	0.0	215

Table 2: Average benchmark scores of various models and agents across select task categories; see Supp. **??** for complete list. Evaluation performed against EXP-Bench.

Category	Agent	Model	D	I	E	I·E	C	All✓	All✓·E
Applications	OH	Nova Pro	19.2	23.9	19.0	0.0	13.9	0.0	0.0
Applications	OH	o3-mini	9.0	8.0	0.0	0.0	8.3	0.0	0.0
Applications	OH	3.5 Haiku	19.2	24.5	8.3	5.6	8.3	0.0	0.0
Applications	OH	3.7 Sonnet	9.0	26.8	30.8	7.7	8.3	2.8	0.0
Applications	IA	Nova Pro	0.0	9.8	5.0	0.0	0.0	0.0	0.0
Applications	IA	3.5 Haiku	0.0	18.0	0.0	0.0	0.0	0.0	0.0
Applications	OH	DeepSeek R1	3.1	4.3	0.0	0.0	0.0	0.0	0.0
RL	OH	3.7 Sonnet	18.3	48.2	27.3	21.2	17.6	2.0	3.0
RL	OH	o3-mini	23.5	34.8	15.7	2.0	27.5	3.9	0.0
RL	OH	3.5 Haiku	27.7	41.4	11.5	0.0	17.6	0.0	0.0
RL	IA	3.5 Haiku	3.3	27.5	17.4	0.0	2.4	0.0	0.0
RL	OH	DeepSeek R1	5.0	10.3	0.0	0.0	2.0	0.0	0.0
RL	IA	Nova Pro	0.0	8.6	9.7	0.0	0.0	0.0	0.0
RL	OH	Nova Pro	17.9	28.9	0.0	0.0	13.7	0.0	0.0

Conclusion without execution. OpenHands + o3-mini achieves C=21.0% despite E=15.0%, suggesting that agents sometimes produce plausible-sounding but unverified conclusions.

DeepSeek R1 struggles overall. OpenHands + DeepSeek R1 achieves the lowest scores across all metrics (D=6.8%, I=10.0%, E=0.7%), suggesting limited capability in the research experimentation domain relative to other frontier models.

IterativeAgent underperforms OpenHands. Despite achieving moderate execution rates (IA + Nova Pro: E=18.1%; IA + Claude-3.5 Haiku: E=25.2%), IterativeAgent scores zero on All✓ and All·E✓ across all configurations, indicating that execution alone is insufficient without correct design and conclusions.

4.4 Category-Level Analysis

Table 2 breaks down performance by research category (Applications and RL shown; see supplementary for the complete list).

RL tasks show relatively stronger performance. OpenHands + Claude-3.7 Sonnet achieves I=48.2% and I·E=21.2% on RL tasks, the highest implementation-execution alignment across all categories and configurations. This may reflect that RL experiment code tends to be more self-contained, with fewer external data dependencies. However, All·E✓ remains low (3.0%), demonstrating the challenge of completing all four phases.

Application tasks are particularly challenging. The Applications category generally sees lower performance, with most configurations achieving All·E✓=0%. Application tasks often involve domain-specific dependencies (e.g., molecular datasets, medical imaging pipelines) that are difficult for agents to set up correctly.

Table 3: Agents fail in diverse ways across different phases of experimentation; this table presents a simplified subset of common examples, measured across all agent and model evaluations.

Phase	Failure Type	Prevalence (%)
Design	Incomplete or Misclassified Design Variables	16.05
Design	Irrelevant Procedural Additions in Design	7.62
Implementation	Missing Essential implementation Components	39.71
Implementation	Incomplete Evaluation Metric Implementation	2.15
Implementation	Incomplete Data and Preprocessing Setup	1.83
Execution	Environment/Dependency Configuration Errors	29.38
Execution	Execution Script and File Errors	23.84
Execution	Missing Setup Script File	6.95
Execution	Tensor Operation Execution Error	3.22
Conclusion	Missing Conclusion Content	26.18
Conclusion	Incorrect Conclusion Interpretation	19.66
Conclusion	Extraneous Details in Conclusion	7.77
Conclusion	Incorrect Numeric Conclusion	3.21

4.5 Failure Analysis

Table 3 presents a taxonomy of failure modes observed across all agent–model evaluations, organized by experimental phase.

Implementation failures dominate. Missing essential implementation components is the most prevalent failure (39.7%), indicating that agents frequently produce incomplete code that omits key algorithmic components. This is often because the masked repository provides partial scaffolding that agents fail to extend correctly.

Execution failures are systematic. Environment/dependency configuration errors (29.4%) and script/file errors (23.8%) together account for most execution failures. These are largely infrastructure-level problems: incorrect package versions, missing CUDA configurations, and relative path errors that prevent otherwise correct code from running.

Conclusion failures reflect evaluation gap. Missing conclusion content (26.2%) and incorrect conclusion interpretation (19.7%) suggest that even when agents execute code, they often fail to correctly report or interpret results. This highlights a critical gap between code execution and scientific interpretation.

Design failures reflect structural misunderstanding. Incomplete or misclassified design variables (16.0%) indicate that agents sometimes misidentify which experimental factors are independent variables versus hyperparameters, leading to fundamentally incorrect experimental designs.

4.6 Conjunctive Scoring Reveals True Performance

Figure 5 illustrates the impact of conjunctive scoring. Average performance across all configurations on monitor-passing tasks (D, I, or C individually) reaches approximately 20.6%, but drops to 0.2% when all four phases are required simultaneously (All·E✓). This order-of-magnitude reduction in apparent performance underscores the necessity of holistic evaluation: agents that appear competent on isolated metrics may be entirely unable to complete real experiments.

4.7 Cost–Time Trade-offs

Figure 6 shows the relationship between agent cost/time and evaluation performance. There is no consistent monotonic relationship between cost and performance: more expensive configurations do not uniformly achieve higher scores. OpenHands + Claude-3.7 Sonnet achieves the highest I and E scores but at moderate cost. IterativeAgent configurations are cheaper but achieve lower overall performance. This suggests that agent scaffold design, not just model capability, is a critical factor.

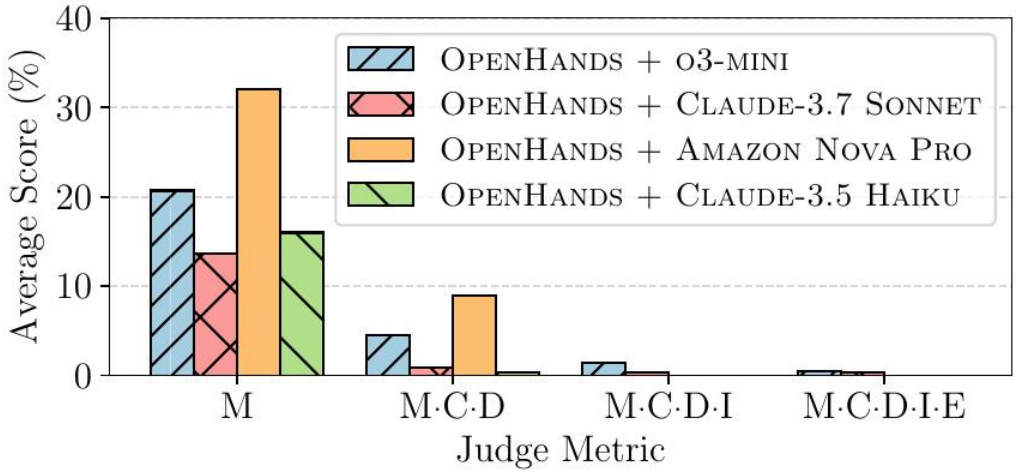

Figure 5: Stricter metrics reveal lower true correctness. Conjunctive scoring dramatically reduces apparent agent performance, showing that partial success on individual phases does not indicate complete experimental capability.

4.8 Stability Analysis

Figure 7 presents a stability analysis of EXP-Bench evaluation scores, confirming that near-zero All·E✓ scores reflect genuine agent limitations rather than evaluation noise.

5 Discussion

The Challenge of End-to-End Experimentation. Our results demonstrate that the bottleneck in AI research experimentation is not any single capability but the *conjunction* of multiple capabilities: understanding experimental design, writing correct and runnable code, setting up complex environments, and correctly interpreting results. Current agents can do each of these to some degree in isolation, but rarely chain them all together successfully. This suggests that future work should focus on integrated agent architectures that maintain coherence across all experimental phases [5, 9].

Environment Setup as a Key Barrier. The high prevalence of environment and dependency errors (29.4%) points to a critical limitation: agents struggle with the "plumbing" of research—installing correct package versions, configuring GPU environments, and managing file paths. This is a systems challenge as much as a reasoning one.

Limitations. EXP-Bench has several limitations. First, the benchmark covers 51 papers from two venues (NeurIPS/ICLR 2024), which may not generalize to all AI research styles. Second, human quality control relies on execution-based validation and spot-checks rather than full expert review of all 461 tasks. Third, the 40-minute execution timeout may penalize computationally intensive experiments that would succeed with more time. Fourth, the LLM judge (o3-mini) may have systematic biases in evaluating design and conclusion quality, though our stability analysis suggests these are minor. Future work should expand the benchmark to additional venues and domains, and conduct more extensive human validation.

Implications for AI Research Automation. EXP-Bench establishes a clear empirical baseline for the current state of AI research automation. While systems like The AI Scientist [20] and Agent Laboratory [22] demonstrate the potential for automated research, our results suggest that reliably replicating and executing specific published experiments remains an unsolved challenge. This is an important prerequisite for trustworthy AI-assisted research.

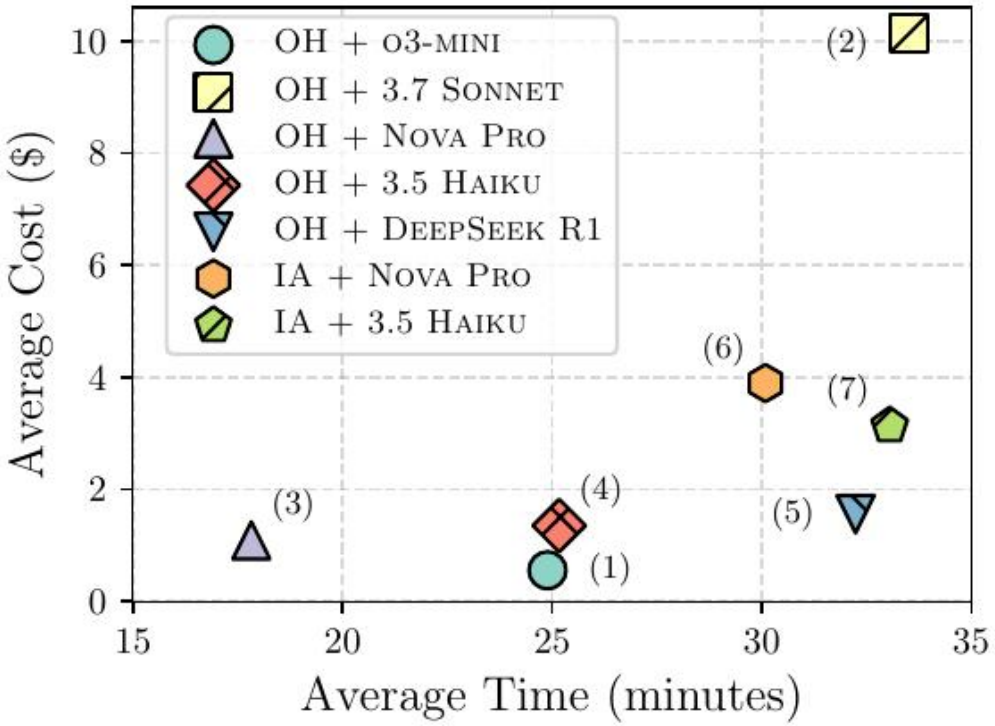

Figure 6: Cost–time trade-offs across agents. More expensive configurations do not consistently achieve higher performance on EXP-Bench metrics.

6 Conclusion

We presented EXP-Bench, the first benchmark for evaluating AI agents on complete, executable AI research experiments. Drawing from 51 NeurIPS/ICLR 2024 papers, EXP-Bench provides 461 tasks with execution-grounded ground truth and a rigorous conjunctive evaluation protocol. Our evaluation of seven agent–model configurations reveals that agents achieve meaningful performance on individual experimental phases but struggle to chain them together: the best All·E✓ score across all configurations is 0.5%, despite individual metric scores reaching 35%. Systematic failure analysis identifies missing implementation components, environment configuration errors, and missing or incorrect conclusions as the dominant bottlenecks.

We hope EXP-Bench will serve as a rigorous testbed for future progress in AI research automation, encouraging the community to develop agents that can reliably conduct complete experiments rather than excelling at isolated sub-tasks. The benchmark, evaluation code, and dataset are publicly available to support this goal.

References

[1] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang. Researchagent: Iterative research idea generation over scientific literature with large language models. *arXiv preprint arXiv:2404.07738*, 2024.

[2] P. Berens, K. Cranmer, N. D. Lawrence, U. von Luxburg, and J. Montgomery. Ai for science: An emerging agenda, 2023.

[3] A. Birhane, A. Kasirzadeh, D. Leslie, and S. Wachter. Science in the age of large language models. *Nature Reviews Physics*, 5(5):277–280, 2023.

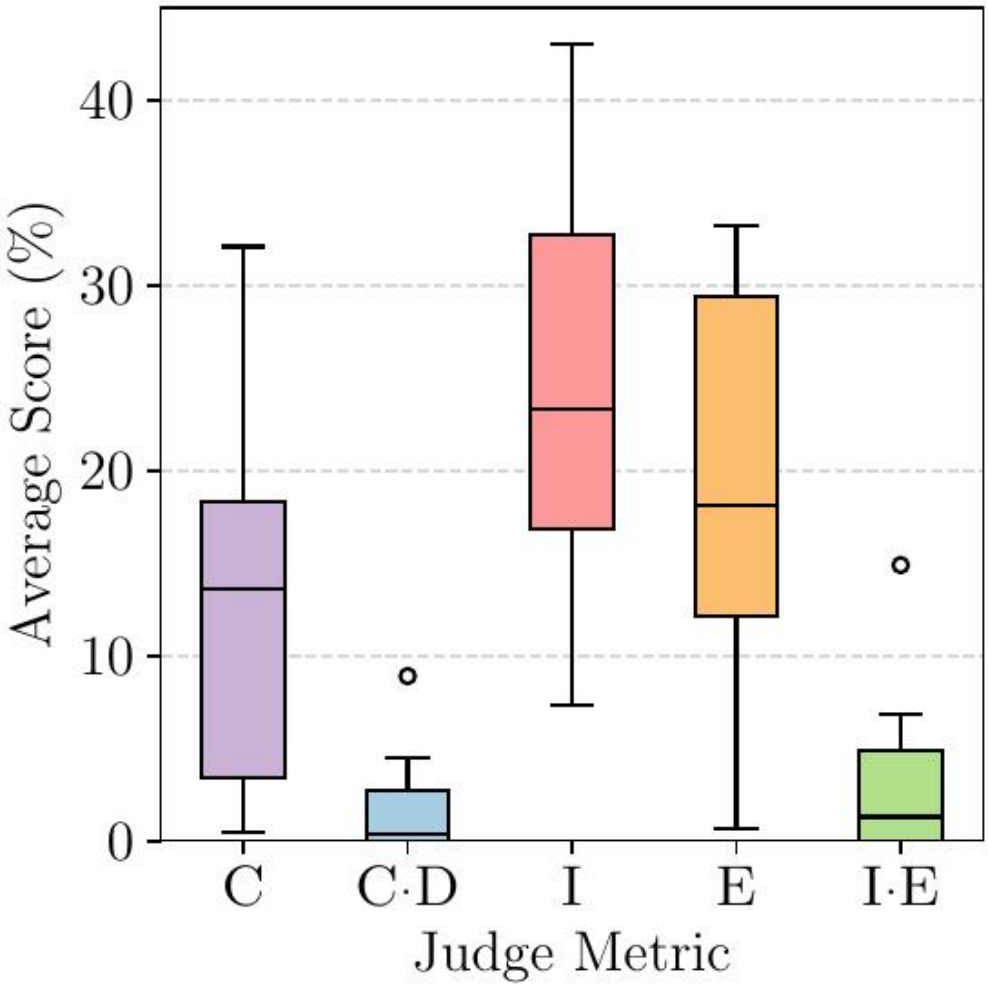

Figure 7: Stability analysis of EXP-Bench evaluation scores across repeated evaluations.

[4] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering, 2024.

[5] A. Fourney, G. Bansal, H. Mozannar, C. Tan, E. Salinas, F. Niedtner, G. Proebsting, G. Bassman, J. Gerrits, J. Alber, et al. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.

[6] A. Ghafarollahi and M. J. Buehler. Sciagents: Automating scientific discovery through multi-agent intelligent graph reasoning, 2024.

[7] K. Gu, R. Shang, R. Jiang, K. Kuang, R.-J. Lin, D. Lyu, Y. Mao, Y. Pan, T. Wu, J. Yu, et al. Blade: Benchmarking language model agents for data-driven science. *arXiv preprint arXiv:2408.09667*, 2024.

[8] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang. Ds-agent: Automated data science by empowering large language models with case-based reasoning. *arXiv preprint arXiv:2402.17453*, 2024.

[9] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. MetaGPT: Meta programming for

a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024.

[10] X. Hu, Z. Zhao, S. Wei, Z. Chai, Q. Ma, G. Wang, X. Wang, J. Su, J. Xu, M. Zhu, et al. Infiagent-dabench: Evaluating agents on data analysis tasks. *arXiv preprint arXiv:2401.05507*, 2024.

[11] Q. Huang, J. Vora, P. Liang, and J. Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation, 2024.

[12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[13] H. Kitano. Nobel turing challenge: creating the engine for scientific discovery. *npj Systems Biology and Applications*, 7(1):29, Jun 2021.

[14] P. T. J. Kon, J. Liu, Q. Ding, Y. Qiu, Z. Yang, Y. Huang, J. Srinivasa, M. Lee, M. Chowdhury, and A. Chen. Curie: Toward rigorous and automated scientific experimentation with ai agents, 2025.

[15] S. Kramer, M. Cerrato, S. Džeroski, and R. King. Automated scientific discovery: From equation discovery to autonomous discovery systems, 2023.

[16] J. Lála, O. O'Donoghue, A. Shtedritski, S. Cox, S. G. Rodriques, and A. D. White. Paperqa: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2023.

[17] S. Li, Z. Wang, Z. Liu, C. Tan, H. Lin, D. Wu, Z. Chen, J. Zheng, and S. Z. Li. Moganet: Multi-order gated aggregation network, 2024.

[18] T. Liu, C. Xu, and J. McAuley. Repobench: Benchmarking repository-level code auto-completion systems, 2023.

[19] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. Agentbench: Evaluating llms as agents, 2023.

[20] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[21] D. Nathani, L. Madaan, N. Roberts, N. Bashlykov, A. Menon, V. Moens, A. Budhiraja, D. Magka, V. Vorotilov, G. Chaurasia, D. Hupkes, R. S. Cabral, T. Shavrina, J. Foerster, Y. Bachrach, W. Y. Wang, and R. Raileanu. Mlgym: A new framework and benchmark for advancing ai research agents, 2025.

[22] S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, Z. Liu, and E. Barsoum. Agent laboratory: Using llm agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.

[23] G. Starace, O. Jaffe, D. Sherburn, J. Aung, J. S. Chan, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson, et al. Paperbench: Evaluating ai's ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.

[24] L. Sun, Y. Han, Z. Zhao, D. Ma, Z. Shen, B. Chen, L. Chen, and K. Yu. Scieval: A multi-level large language model evaluation benchmark for scientific research. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19053–19061, Mar. 2024.

[25] M. Tian, L. Gao, S. D. Zhang, X. Chen, C. Fan, X. Guo, R. Haas, P. Ji, K. Krongchon, Y. Li, S. Liu, D. Luo, Y. Ma, H. Tong, K. Trinh, C. Tian, Z. Wang, B. Wu, Y. Xiong, S. Yin, M. Zhu, K. Lieret, Y. Lu, G. Liu, Y. Du, T. Tao, O. Press, J. Callan, E. Huerta, and H. Peng. Scicode: A research coding benchmark curated by scientists, 2024.

[26] X. Wang, Z. Hu, P. Lu, Y. Zhu, J. Zhang, S. Subramaniam, A. R. Loomba, S. Zhang, Y. Sun, and W. Wang. Scibench: Evaluating college-level scientific problem-solving abilities of large language models, 2024.

[27] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[28] H. Wijk, T. Lin, J. Becker, S. Jawhar, N. Parikh, T. Broadley, L. Chan, M. Chen, J. Clymer, J. Dhyani, E. Ericheva, K. Garcia, B. Goodrich, N. Jurkovic, M. Kinniment, A. Lajko, S. Nix, L. Sato, W. Saunders, M. Taran, B. West, and E. Barnes. Re-bench: Evaluating frontier ai rd capabilities of language model agents against human experts, 2024.

[29] J. Yuan, X. Yan, B. Shi, T. Chen, W. Ouyang, B. Zhang, L. Bai, Y. Qiao, and B. Zhou. Dolphin: Closed-loop open-ended auto-research through thinking, practice, and feedback, 2025.